UPTD BALAI LATIHAN KERJA (BLK) PROVINSI GORONTALO

Pelatihan Berbasis Kompetensi

JUNIOR WEB DEVELOPER

MODUL KERJA 2

Halaman Dinamis dengan JavaScript

Bagian A: HTML, CSS, JavaScript

Batch #5 · Kelas: Junior Web Developer

Instruktur: Rahmat S.A.F. Badaru

2026

Daftar Isi

- A. Identitas Modul
- B. Pemetaan Unit Kompetensi dan Kriteria Capaian
- C. Tujuan Pembelajaran
- D. Petunjuk Belajar
- Kegiatan Belajar 2.1: Dasar JavaScript, Input-Output, dan Percabangan (Hari 5)
- Kegiatan Belajar 2.2: Perulangan, Fungsi, Array, dan Objek (Hari 6)
- Kegiatan Belajar 2.3: DOM, Event, dan Debugging (Hari 7)
- Kegiatan Belajar 2.4: Render Dinamis dan Proyek Versi 2 (Hari 8)
- E. Lembar Kerja Pendamping
- F. Lembar Asesmen Modul 2
- G. Glosarium
- H. Referensi
- Lampiran A: Kode Acuan Kopi Nusantara Versi 2
- Lampiran B: Daftar Periksa Proyek Tema Sendiri Versi 2

A. Identitas Modul

Komponen	Keterangan
Nama modul	Halaman Dinamis dengan JavaScript
Bagian	A: HTML, CSS, JavaScript (Modul 2 dari 3)
Durasi	32 JP (1 JP = 45 menit), 4 hari × 8 JP, **Hari 5-8**
Prasyarat	Modul 1 selesai: proyek Kopi Nusantara Versi 1 dan halaman statis bertema sendiri
Luaran	(1) Proyek latihan bersama "Kopi Nusantara Versi 2"; (2) halaman bertema sendiri Versi 2 (menu, kartu, filter, dan formulir dari data); (3) latihan program JavaScript di folder `latihan`
Perangkat	Komputer atau laptop, koneksi internet minimal 5 Mbps untuk tiap peserta
Perangkat lunak	Laragon, Visual Studio Code, browser (Chrome, Edge, atau Firefox) dengan DevTools

Gambaran modul. Pada Modul 1, halaman Anda bersifat **statis**: isi kartu ditulis langsung di HTML. Pada Modul 2, isi halaman dibuat oleh **JavaScript dari data**. Cukup mengubah data, dan tampilan ikut berubah. Alurnya dimulai dari dasar pemrograman (variabel, percabangan, perulangan, fungsi, array), lalu diterapkan untuk memanipulasi halaman web.

Hari	Fokus	Luaran
5	Dasar JavaScript, input-output, percabangan	Kalkulator belanja sederhana
6	Perulangan, fungsi, array, objek	Program keranjang belanja
7	DOM, event, dan debugging	Halaman latihan interaktif
8	Render dari data, filter, formulir, asesmen	Proyek Versi 2

B. Pemetaan Unit Kompetensi dan Kriteria Capaian

Modul ini melatihkan lima Unit Kompetensi (UK). UK 1.5 (Mengimplementasikan pemrograman terstruktur) menjadi porsi terbesar.

UK	Kode Unit	Nama Unit Kompetensi	JP di Modul 2
1.1	J.620100.005.02	Mengimplementasikan user interface	6
1.2	J.620100.010.01	Menerapkan perintah eksekusi bahasa pemrograman	3
1.3	J.620100.015.01	Menyusun fungsi, file, atau sumber daya pemrograman dalam organisasi yang rapi	2
1.4	J.620100.016.01	Menulis kode dengan prinsip sesuai guidelines dan best practices	2
1.5	J.620100.017.02	Mengimplementasikan pemrograman terstruktur	19
		Jumlah	**32**

Kode	Kriteria capaian	Dilatihkan di
1.1-K3	Mengimplementasikan rancangan UI menjadi halaman yang berinteraksi (menu, filter, formulir)	KB 2.3, 2.4
1.2-K1	Cara dan tools untuk mengeksekusi source code JavaScript diidentifikasi (Console, `<script>`, DevTools)	KB 2.1, 2.3
1.2-K2	Source code dieksekusi sesuai mekanisme (berkas eksternal, urutan pemuatan script)	KB 2.1, 2.4
1.2-K3	Hasil eksekusi diidentifikasi sesuai skenario yang direncanakan (Console, tampilan halaman)	KB 2.1, 2.3
1.3-K1	Nama variabel, fungsi, dan berkas dibuat sesuai konteks	KB 2.2, 2.4
1.3-K2	Source code dilengkapi komentar sesuai konteks	KB 2.2, 2.4
1.3-K3	Folder dan subfolder (`js/`) serta pemisahan `data.js` dan `app.js` sesuai isinya	KB 2.4
1.4-K1	Kode JavaScript ditulis sesuai guidelines dan best practices	KB 2.2, 2.4
1.4-K2	Struktur program terorganisasi (fungsi terpisah untuk data, tampilan, event, inisialisasi)	KB 2.2, 2.4
1.4-K3	Kemudahan interaksi diimplementasikan (validasi formulir dengan pesan jelas, tombol aktif terlihat)	KB 2.4
1.5-K1	Membuat program baca tulis sederhana (input, proses, output)	KB 2.1
1.5-K2	Menerapkan struktur kontrol percabangan dan perulangan	KB 2.1, 2.2
1.5-K3	Menerapkan prosedur dan fungsi sesuai aturan penulisan program	KB 2.2
1.5-K4	Menerapkan array dalam pembuatan program	KB 2.2, 2.4
1.5-K6	Memperbaiki permasalahan pada program	KB 2.3

Kriteria 1.5-K5 (program yang mengakses data dari media penyimpanan) dilatihkan di Modul 3 melalui berkas JSON.

C. Tujuan Pembelajaran

Setelah menyelesaikan modul ini, Anda mampu:

1. Menyusun spesifikasi dan algoritma sederhana (input, proses, output) sebelum menulis program.
2. Menulis dan menjalankan program JavaScript dengan variabel, tipe data, operator, input, dan output.
3. Menerapkan percabangan (`if`, `else`, `switch`) dan perulangan (`for`, `while`).
4. Membuat dan memakai fungsi, array, dan objek.
5. Memilih dan mengubah elemen halaman lewat DOM, serta menanggapi event pengguna.
6. Mengenali jenis kesalahan program dan memperbaikinya dengan Console DevTools.
7. Membuat halaman yang dirender dari data: menu, kartu produk, filter kategori, dan validasi formulir.
8. Menjelaskan secara garis besar ketentuan hukum yang berkaitan dengan pengembangan web (UU ITE, keterbukaan informasi publik, pelindungan data pribadi).

D. Petunjuk Belajar

- Ikuti KB secara berurutan. Setiap KB memuat **Materi**, **Praktik**, **Latihan**, **Rangkuman**, dan **Cek Kemampuan**.
- **Ketik sendiri** setiap kode. Membaca kode tidak sama dengan menulisnya, dan kesalahan kecil justru mengajarkan banyak hal.
- **Buka Console (`F12`) sepanjang belajar.** Console adalah teman utama pemrogram: pesan kesalahan muncul di sana.
- Simpan (`Ctrl+S`) dan segarkan browser (`F5`) setelah setiap perubahan.
- Jika program tidak berjalan: baca pesan kesalahan **dari atas**, catat nama berkas dan nomor barisnya, lalu periksa baris itu.
- Bekerjalah dalam langkah kecil: tulis sedikit, jalankan, pastikan benar, lalu lanjutkan.
- Kode acuan lengkap ada di **Lampiran A**. Gunakan untuk membandingkan setelah Anda mencoba sendiri.

Kegiatan Belajar 2.1: Dasar JavaScript, Input-Output, dan Percabangan

Hari 5 · 8 JP · UK 1.5 (utama) dan 1.2

Tujuan KB 2.1. Setelah mengikuti KB ini, Anda mampu menyusun spesifikasi program sederhana, menjalankan JavaScript, memakai variabel, tipe data, dan operator, membuat program dengan input dan output, serta menerapkan percabangan.

Kegiatan	JP
Algoritma dan spesifikasi program (input, proses, output)	1
Cara menjalankan JavaScript: Console, `<script>`, berkas eksternal	1
Variabel, tipe data, dan operator	2
Input dan output sederhana	1
Percabangan: `if`, `else`, `switch`	2
Latihan dan cek kemampuan	1
Jumlah	**8**

Materi 1: Algoritma dan Spesifikasi Program

Sebelum menulis kode, pikirkan dulu **apa yang ingin dicapai** dan **langkah-langkahnya**.

- **Algoritma** adalah urutan langkah yang jelas dan berhingga untuk menyelesaikan suatu masalah.
- **Spesifikasi program** menjelaskan apa yang masuk, apa yang dikerjakan, dan apa yang keluar. Cara mudahnya dengan tabel **IPO** (*Input, Proses, Output*).

Contoh: kalkulator belanja Kopi Nusantara.

Input	Proses	Output
Harga per bungkus, jumlah bungkus	Total = harga × jumlah. Diskon 10% jika total ≥ 200.000; diskon 5% jika total ≥ 100.000	Total, diskon, total bayar

Pseudocode (algoritma dalam bahasa sehari-hari yang terstruktur):

```
MULAI
  BACA harga, jumlah
  total = harga x jumlah
  JIKA total >= 200000 MAKA diskon = 10% dari total
  JIKA TIDAK, JIKA total >= 100000 MAKA diskon = 5% dari total
  JIKA TIDAK diskon = 0
  totalBayar = total - diskon
  TAMPILKAN total, diskon, totalBayar
SELESAI
```

Flowchart menggambarkan algoritma dengan simbol. Simbol dasarnya:

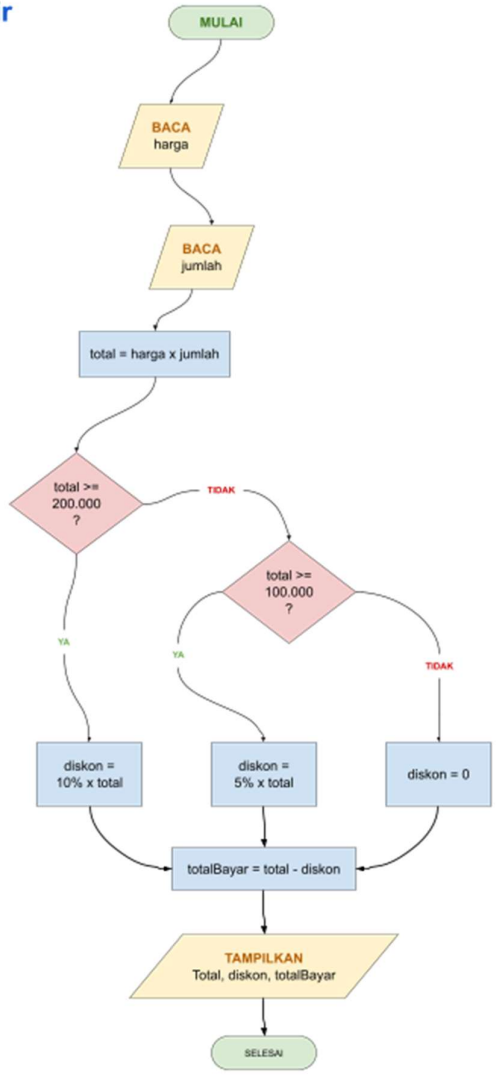
Simbol	Nama	Fungsi
Oval	Terminator	Awal (Mulai) dan akhir (Selesai)

Simbol	Nama	Fungsi
Jajar genjang	Input/Output	Membaca atau menampilkan data
Persegi panjang	Proses	Perhitungan atau penugasan nilai
Belah ketupat	Keputusan	Pertanyaan ya/tidak (percabangan)
Panah	Alur	Arah langkah berikutnya

Algoritma *Pseudocode*

```
MULAI
  BACA harga, jumlah
  total = harga x jumlah
  JIKA total >= 200000 MAKA diskon = 10% dari total
  JIKA TIDAK, JIKA total >= 100000 MAKA diskon = 5% dari total
  JIKA TIDAK diskon = 0
  totalBayar = total - diskon
  TAMPILKAN total, diskon, totalBayar
SELESAI
```

Bagan Alir



Materi 2: Menjalankan JavaScript

JavaScript adalah bahasa pemrograman yang berjalan di browser. Ada tiga cara menjalankannya:

Cara	Keterangan
Console DevTools	Ketik kode langsung di tab *Console* (`F12`); cocok untuk mencoba cepat
`<script>` internal	Kode ditulis di dalam HTML: `<script> ... </script>`
Berkas eksternal	`<script src="js/app.js"></script>`. **Dipakai di pelatihan ini**, sejalan dengan CSS eksternal

Letakkan tag `<script>` **di akhir** `<body>` (sebelum `</body>`) agar seluruh elemen HTML sudah dimuat saat kode dijalankan. Browser membaca script **dari atas ke bawah**, sehingga urutan pemuatan penting.

Praktik 1: Halaman Latihan JavaScript

Di dalam folder `latihan`, buat dua berkas berikut.

`js-dasar.html`:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Latihan JavaScript Dasar</title>
</head>
<body>
  <h1>Latihan JavaScript Dasar</h1>
  <p>Buka Console (F12) untuk melihat hasilnya.</p>
  <script src="js-dasar.js"></script>
</body>
</html>
```

`js-dasar.js`:

```
console.log("Halo, Kopi Nusantara!");
```

Buka `js-dasar.html` di browser lewat Laragon, tekan `F12`, dan lihat tab *Console*. Tulisan "Halo, Kopi Nusantara!" muncul di sana. Inilah langkah pertama **mengeksekusi source code** dan **mengidentifikasi hasilnya**.

Materi 3: Variabel, Tipe Data, dan Operator

Variabel adalah wadah bernama untuk menyimpan nilai.

```
const namaProduk = "Kopi Gayo";  // nilai tetap: gunakan const
let stok = 20;                   // nilai bisa berubah: gunakan let
stok = stok - 1;                 // stok sekarang 19
console.log(stok);                 // stok sekarang 19
```

- `const` untuk nilai yang tidak diubah, `let` untuk nilai yang berubah. **Hindari `var`.**
- Aturan nama: diawali huruf, tanpa spasi, **peka huruf besar-kecil** (`stok` berbeda dari `Stok`). Gunakan gaya **camelCase**: `hargaPerBungkus`, `totalBayar`.

Tipe data dasar:

Tipe	Contoh	Keterangan
string	`"Kopi Toraja"`	Teks, ditulis di dalam tanda kutip
number	`85000`, `3.5`	Bilangan bulat maupun desimal
boolean	`true`, `false`	Benar atau salah
undefined	`undefined`	Variabel ada tetapi belum diberi nilai
null	`null`	Sengaja dikosongkan

Periksa tipe dengan `typeof`, misalnya `typeof 85000` menghasilkan `"number"`.

Operator:

Jenis	Operator	Contoh
Aritmetika	`+ - * / %`	`10 % 3` menghasilkan `1` (sisa bagi)
Penugasan	`= += -= ++ --`	`stok -= 1` sama dengan `stok = stok - 1`
Perbandingan	`=== !== > < >= <=`	`harga >= 100000` menghasilkan `true` atau `false`

Operator logika menggabungkan kondisi:

- `&&` berarti **DAN** (keduanya benar), misalnya `harga > 0 && jumlah > 0`.
- `||` berarti **ATAU** (salah satu benar).
- `!` berarti **BUKAN** (membalik nilai).

Penting: gunakan `===` (sama dengan, termasuk tipenya), bukan `==`. Contoh: `"5" === 5` bernilai `false`.

Menggabungkan teks: dengan tanda `+` atau *template literal* (tanda kutip terbalik dan `${...}`):

```
const nama = "Kopi Gayo";
const harga = 80000;
console.log("Produk: " + nama + ", harga: Rp " + harga);
console.log(`Produk: ${nama}, harga: Rp ${harga}`);
```

Materi 4: Input dan Output Sederhana

Fungsi	Kegunaan
`console.log()`	Menampilkan hasil di Console (untuk pemrogram)
`alert()`	Menampilkan kotak pesan
`prompt()`	Meminta pengguna mengetik sesuatu; hasilnya **selalu berupa teks (string)**
`Number()`	Mengubah teks menjadi angka

Karena `prompt()` menghasilkan teks, ubah dulu menjadi angka sebelum menghitung:

```
const jumlah = Number(prompt("Jumlah bungkus:"));
```

Angka dapat ditampilkan dengan format Indonesia: `85000..toLocaleString("id-ID")` menghasilkan `85.000`, atau lebih lazim `angka.toLocaleString("id-ID")` pada variabel.

`prompt()` dan `alert()` dipakai hanya untuk belajar. Pada halaman web sungguhan, input dan output dilakukan lewat elemen halaman (dipelajari di KB 2.3).

Materi 5: Percabangan

Percabangan membuat program **memilih jalur** berdasarkan kondisi.

```
const total = 150000;

if (total >= 200000) {
  console.log("Diskon 10%");
} else if (total >= 100000) {
  console.log("Diskon 5%");
} else {
  console.log("Tidak ada diskon");
}
```

`switch` cocok jika membandingkan satu nilai dengan beberapa pilihan:

```
const kategori = "arabika";

switch (kategori) {
  case "arabika":
    console.log("Rasa lebih asam dan beraroma");
    break;
  case "robusta":
    console.log("Rasa lebih pahit dan kafein tinggi");
    break;
  default:
    console.log("Kategori tidak dikenal");
}
```

Jangan lupa `break` pada setiap `case`, dan `default` untuk nilai di luar pilihan.

Praktik 2: Kalkulator Belanja

Buat `latihan/kalkulator.html` (salin kerangka HTML dari Praktik 1, ganti `src` menjadi `kalkulator.js`) dan `latihan/kalkulator.js`:

```
// Kalkulator belanja Kopi Nusantara

// 1. Input
const hargaPerBungkus = Number(prompt("Harga per bungkus (Rp):"));
const jumlah = Number(prompt("Jumlah bungkus:"));

// 2. Proses
const total = hargaPerBungkus * jumlah;
let diskon = 0;

if (total >= 200000) {
  diskon = total * 0.1;
} else if (total >= 100000) {
  diskon = total * 0.05;
}

const totalBayar = total - diskon;

// 3. Output
console.log(`Total      : Rp ${total.toLocaleString("id-ID")}`);
```

```
console.log(`Diskon     : Rp ${diskon.toLocaleString("id-ID")}`);
console.log(`Total bayar: Rp ${totalBayar.toLocaleString("id-ID")}`);
alert(`Total bayar: Rp ${totalBayar.toLocaleString("id-ID")}`);
```

Uji dengan beberapa nilai. Contoh: harga 85000 dan jumlah 3 menghasilkan total 255.000, diskon 25.500, total bayar 229.500. Uji juga batas: total tepat 100.000 dan 200.000. Program yang baik **diuji pada nilai batas**.

Menangani input yang salah. Bagaimana jika pengguna mengetik huruf atau angka nol? Tambahkan pemeriksaan **sebelum** proses:

```
if (isNaN(hargaPerBungkus) || isNaN(jumlah) || hargaPerBungkus <= 0 || jumlah <= 0) {
  alert("Harga dan jumlah harus berupa angka lebih dari 0.");
} else {
  // ... proses dan output di sini ...
}
```

`isNaN(x)` bernilai `true` jika `x` bukan angka.

Latihan 1

1. Buat program yang meminta sebuah angka, lalu menampilkan apakah angka itu **genap** atau **ganjil** (petunjuk: operator `%`).
2. Buat program penentu **grade**: nilai 90 ke atas = A, 80 ke atas = B, 70 ke atas = C, selain itu D.
3. Buat spesifikasi IPO dan pseudocode untuk salah satu program di atas **sebelum** menulis kodenya.
4. Tambahkan validasi pada kalkulator belanja agar nilai negatif ditolak dengan pesan yang jelas.
5. *Tantangan:* tambahkan ongkos kirim Rp 10.000 jika total bayar kurang dari Rp 100.000.

Rangkuman KB 2.1

- Algoritma dan spesifikasi IPO disusun sebelum menulis kode.
- JavaScript dijalankan lewat Console atau berkas eksternal (`<script src>`) di akhir `<body>`.
- Variabel: `const` untuk nilai tetap, `let` untuk nilai berubah. Gunakan `===` untuk perbandingan.
- `prompt()` menghasilkan teks, sehingga perlu `Number()` sebelum dihitung.
- Percabangan: `if`, `else if`, `else`, dan `switch`.

Cek Kemampuan KB 2.1

- ☐ Saya dapat menyusun tabel IPO dan pseudocode untuk masalah sederhana.
- ☐ Saya dapat menjalankan JavaScript dari berkas eksternal dan membaca hasilnya di Console.
- ☐ Saya dapat menjelaskan perbedaan `const` dan `let`.
- ☐ Kalkulator belanja saya berjalan dan sudah diuji pada nilai batas.
- ☐ Saya dapat menulis `if`, `else if`, `else` tanpa melihat contoh.

Kesalahan umum: menulis `=` (penugasan) padahal maksudnya `===` (perbandingan); lupa mengubah hasil `prompt()` menjadi angka (`"5" + "3"` menghasilkan `"53"`); tanda kutip atau kurung tidak berpasangan; lupa `break` pada `switch`.